

# 数据方体格

---

# 数据方体格结构

---

- 多维数据模型可以用一个数据方体来表示，一个数据方体由多个维和度量组成。
- 由特定维和度量组成一个**cuboid**或一个视图。给定一组维，则由不同维或维上的不同层可以组合出多个**cuboid**。
- 例如，假设数据集有A、B、C三个维，每个维上只有一个层，则可以组合出8个**cuboid**，分别是：ABC，AB，AC，BC，A，B，C，all。

- 
- 一般地讲，对于一个 $n$ 维的数据集，如果不考虑层次的话，它的cuboid总数是 $2^n$
  - 其中包含维的个数最多的那个cuboid就是事实表，通常被称作base cuboid (或base view) 。
  - 包含维的个数最少的那个cuboid通常被称为apex cuboid (或apex view)，一般用all表示。

# 导出关系

---

- 如果cuboid a是由cuboid b通过减少维的个数或者上升维的层得到的，则称cuboid a可以由cuboid b导出。
- 如，cuboid AB, cuboid BC, cuboid AC可以由cuboid ABC导出。cuboid A, cuboid B可以由cuboid AB导出。

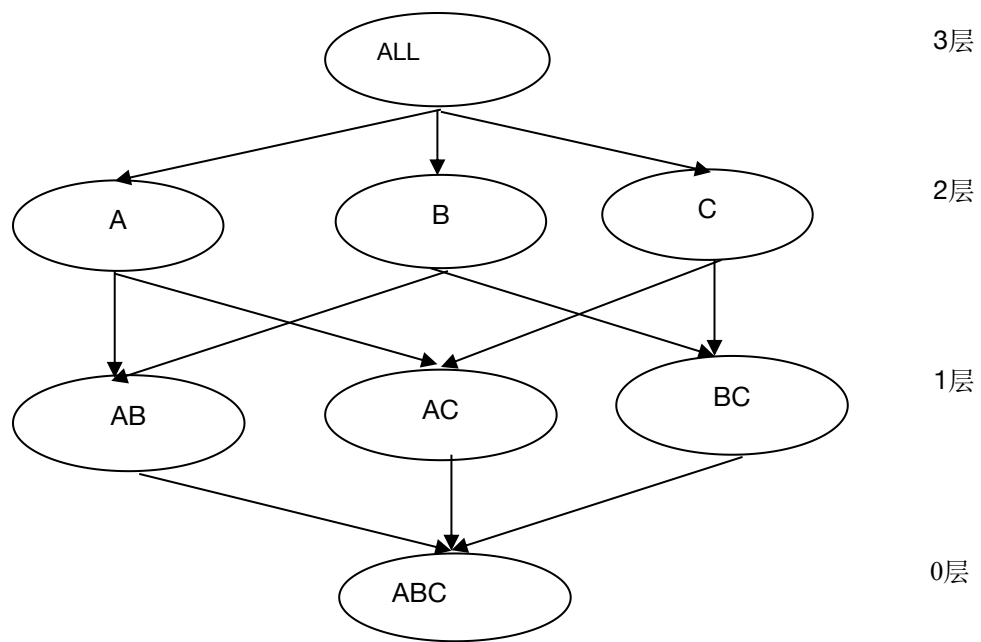


图5.1 数据方体格示例

- 
- 根据导出关系，可以将所有的**cuboid**组织成一个格结构
  - 这样的一个个格结构称为一个数据方体格，有时也简称为数据方体。
  - 其中层次最低的**cuboid**就是**base cuboid**，层次最高的**cuboid**就是**ALL**即**apex cuboid**。

- 
- 数据方体格结构实际上是一个有向图，图中的每个结点表示一个**cuboid**，图中的每条边表示结点之间的导出关系。
  - 通常用一个二元组( $L, \leq$ )来表示，其中 $L$ 表示格中所有的**cuboid**， $\leq$ 表示**cuboid**间的导出关系。

# Data compression

---

- Dwarf
- Condensed cube
- Quotient Cube

# 语义压缩

---

- 一类新的有效的数据方体存储结构--按照数据方体中数据所代表的不同的语义(或模式)信息将数据进行划分后压缩存储。
  - **Drawf**: 利用数据的前缀和后缀语义信息将数据方体进行压缩
  - **Condensed**: 利用单元组语义信息对数据方体进行压缩
  - **Quotient**: 利用聚集值相等且连通的语义信息对数据方体进行压缩。

# Dwarf

---

- **Dwarf**数据方体将具有相同前缀或后缀的数据元素压缩存储在一起。
- 一般来说，数据方体中数据密集的地方前缀冗余比较多，数据稀疏的地方后缀冗余比较多。
- **Drawf**数据方体在存储时消除了这两种类型的冗余信息，大大缩减了数据方体的存储空间。将一个完全实体化的数据方体缩减到一个非常密集的数据结构中。

# Dwarf

---

- 前缀冗余是指某个维值或某几个维值的组合会同时出现多个视图中的前缀中
- 后缀冗余是指多个视图共享相同的后缀

# Dwarf

| Store | Customer | Product | Price |
|-------|----------|---------|-------|
| S1    | C2       | P2      | \$70  |
| S1    | C3       | P1      | \$40  |
| S2    | C1       | P1      | \$90  |
| S2    | C1       | P2      | \$50  |

前缀冗余: S1出现7次:

<S1, C2, P2>,  
<S1, C3, P1>,  
<S1, C2>,  
<S1, C3>,  
<S1, P2>,  
<S1, P1>,  
<S1>

后缀冗余: C1总是和S2同时出现。

# Dwarf

---

- Dwarf数据方体将这两种类型的冗余提取出来，在数据方体中只存储一次
- 好处：节约存储空间，缩短计算时间

# Dwarf

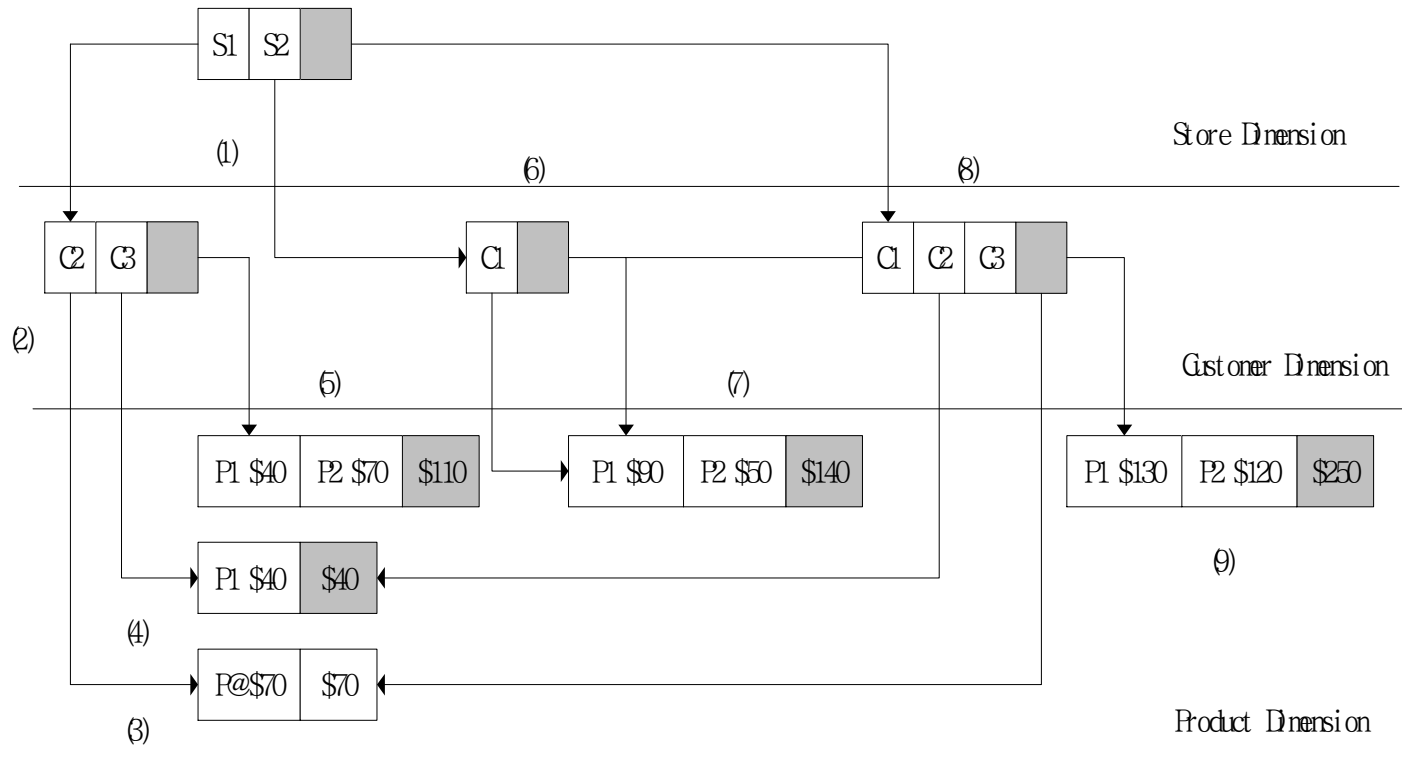


图3-16 一个Dwarf数据方体的例子

# Dwarf

---

- Drawf结构的高度等于维的个数，每个维对应于该图中的一层

# Dwarf

- 根结点包含若干个二元组，每个二元组是形如[关键字：指针]的对，每个二元组对应于一个维值
- 二元组[K,P]中的指针P指向下面一层中的某个结点，该结点和关键字K相关联
- 每个非叶子结点都有一个特殊维值ALL，如结点中右边的灰色区域所示，它指向下一维的所有维值
- 被某个二元组[K,P]的指针所指的结点以及其中的所有二元组都由K所控制
  - 例如：关键字S1所在的二元组控制了关键字C2和C3所在的结点

# Dwarf

- 从根到叶子结点的一条路经，如， $\langle S1, C3, P1 \rangle$ ，对应于视图 $\langle \text{Store}, \text{Customer}, \text{Product} \rangle$ 中的一个元素
- 该路经指向一个结点 $[P1 \$40]$ ，该结点包含了 $\langle S1, C3, P1 \rangle$ 的聚集值
- 其他的一些路径可以使用特殊值ALL，如 $\langle S2, ALL, P2 \rangle$ 指向结点 $[P2 \$50]$ ，对应于由所有的顾客在商店S2所购买的产品P2的总和
- 在叶子层，每个结点的二元组具有[关键字：聚集值]的形式。它表示了顺着从根到该叶子结点所在的路经上所有元素的聚集值

# Dwarf

---

- 每个叶子结点也有一个特殊值**ALL**，存储该结点所有的元素的聚集值
- 路径<**ALL, ALL,ALL**>指向所有元组的聚集值之和

# Dwarf

- Dwarf通过将结点合并存储，避免了冗余存储
  - 如，顺着 $\langle S2, C1, P2 \rangle$ ,  $\langle S2, ALL, P2 \rangle$ 和 $\langle ALL, C1, P2 \rangle$ 所得到的聚集值,全是从表2-3中的最后一条元组得到的，它们都指向相同的结点[P2 \$50]。如果这些结点分别存储的话,将存在后缀冗余
- 凡是多个指针所指的结点都是合并存储的结点

# Dwarf

---

- 对Drawf进行遍历时，需要从根走到叶子，该路径的长度永远为D
- 一条路径中的第i个值要么是第i维的一个维值，要么是特殊维值ALL
- Drawf结构本身就构成了一个有效的内部索引结构，所以不再需要额外的索引

# Condensed Cube

- 如果某个元组是从基本表中的某个元组直接得到的，则不存储该元组

| A | B | C | M  |
|---|---|---|----|
| 0 | 0 | 0 | 10 |
| 0 | 0 | 1 | 30 |
| 0 | 1 | 1 | 20 |
| 0 | 1 | 2 | 60 |
| 0 | 2 | 3 | 40 |

| View | A | B | C | M  |
|------|---|---|---|----|
| ABC  | 0 | 0 | 0 | 10 |
|      | 0 | 0 | 1 | 30 |
|      | 0 | 1 | 1 | 20 |
|      | 0 | 1 | 2 | 60 |
|      | 0 | 2 | 3 | 40 |
| AB   | 0 | 0 | * | 40 |
|      | 0 | 1 | * | 80 |
|      | 0 | 2 | * | 40 |
| AC   | 0 | * | 0 | 10 |
|      | 0 | * | 1 | 50 |
|      | 0 | * | 2 | 60 |
|      | 0 | * | 3 | 40 |

| View | A | B | C | M   |
|------|---|---|---|-----|
| BC   | * | 0 | 0 | 10  |
|      | * | 0 | 1 | 30  |
|      | * | 1 | 1 | 20  |
|      | * | 1 | 2 | 60  |
|      | * | 2 | 3 | 40  |
| A    | 0 | * | * | 160 |

| View | A | B | C | M   |
|------|---|---|---|-----|
| B    | * | 0 | * | 40  |
|      | * | 1 | * | 80  |
|      | * | 2 | * | 40  |
| C    | * | * | 0 | 10  |
|      | * | * | 1 | 50  |
|      | * | * | 2 | 60  |
|      | * | * | 3 | 40  |
| {}   | * | * | * | 160 |

# Condensed Cube

---

- **Condensed**数据方体：利用了数据方体中存在着一些数据元素是从同一个基元组（事实表中的元组）聚集而来（从而具有相同的聚集值）的特点，将多个数据元素压缩存放在一个物理单元中，从而节省了大量的存储空间。

# Condensed Cube

---

- 和其他的压缩方法相比，**Condensed**有如下好处：
  - 回答查询时不需要解压缩，且可以精确地回答查询。有些数据压缩是通过近似的方法得到的，如抽样，直方图等，因此只能得到近似的查询结果
  - 完全实体化，查询时不需要进一步的聚集计算。有别于只实体化一部分视图来减少数据方体大小的方法。
  - 适用于所有的OLAP应用。有别于那些压缩后的数据只能用来回答某种类型的特殊查询的方法。

# 一个极端的例子

- 假设事实表R只有一个元组  $(a_1, a_2, \dots, a_n, V)$
- 由R计算出来的数据方体有 $2^n$ 个数据元素:
  - $(a_1, a_2, \dots, a_n, V_1)$ ,
  - $(a_1, *, \dots, *, V_2)$ ,
  - $(*, a_2, *, \dots, a_n, V_3)$ ,
  - $(*, *, \dots, *, V_m)$
- 实际上,  $V_1 = V_2 = \dots = V_m = \text{agg}(r) = V$ 。
- 显然, 物理上只需要存储元组  $(a_1, a_2, \dots, a_n, V)$  即可, 其他的数据元素都具有相同值。
- 当查询这些数据元素的时候, 只需要直接返回V的值, 而不需要再进行聚集计算。

# Condensed Cube

---

- 计算Condensed Cube的关键是:识别出其中的单元组, 将在其上的所有数据元素压缩存储。
- **重要原理:** 如果某个元组 $r$ 是单维集合 $SD$ 上的单元组, 则 $r$ 一定是 $SD$ 的任何超集上的单元组。
  - 例如, 如果元组  $(0, 1, 1, 50)$  是属性 $A$ 上的单元组, 它肯定是属性 $\{A\}$ ,  $\{AB\}$ ,  $\{AC\}$ ,  $\{ABC\}$ 上的单元组。

# Condensed Cube

- 符号**SDSET**：用来代表与一个单元组相关联的多个维属性集的集合。
- 根据单元组和它的**SDSET**，就可以得出由该单元组聚集而成的可以加以压缩的多个数据元素。
- 例如：已知单元组 (0, 1, 1, 50)和它的维属性集集合**SDSET**= $\{\{A\}, \{AB\}, \{AC\}, \{ABC\}\}$ ，可以得出下列数据元素： (0, \*, \*, 50) , (0, 1, \*, 50) , (0, \*, 1, 50) , (0, 1, 1, 50)具有相同的聚集值，可以压缩到一个物理单元中。

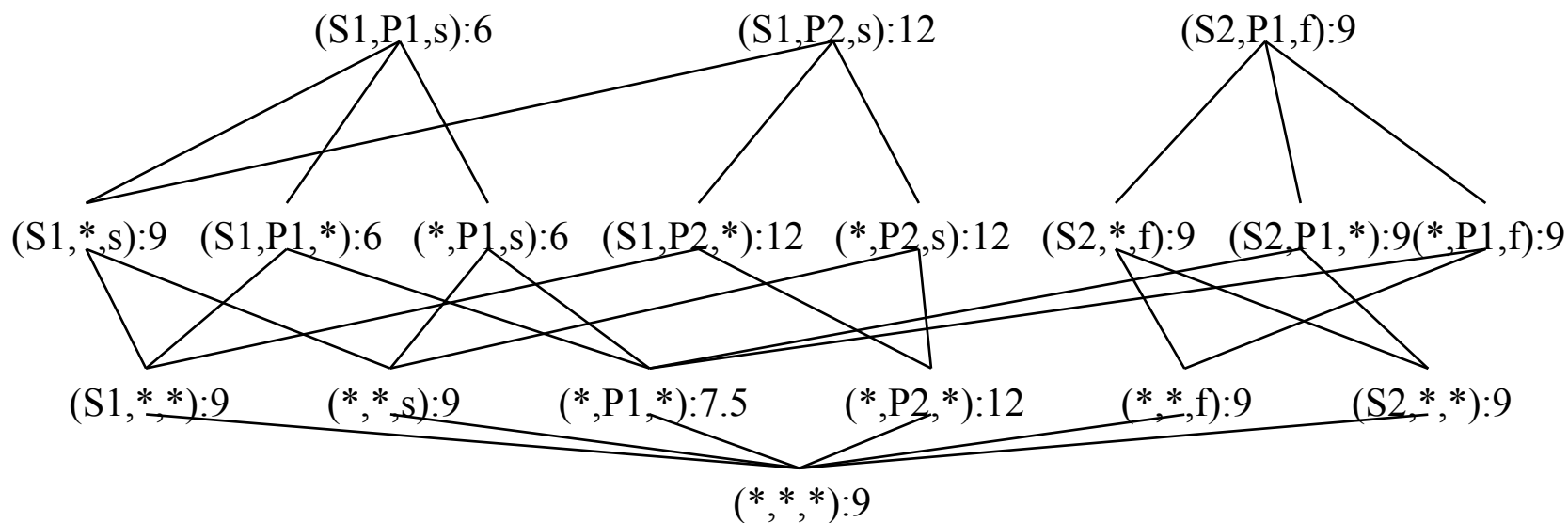
# Condensed Cube

---

- 查询时，这些数据元素可以由单元组**恢复 (Expand)** 出来。
- **Expand**操作：输入是单元组和**SDSET**，输出是一个或一组数据元素：
- **Expand**操作非常简单，只需要拷贝单元组，然后将某些属性设置成特殊值**all**即可，并不需要进一步的聚集或计算操作。

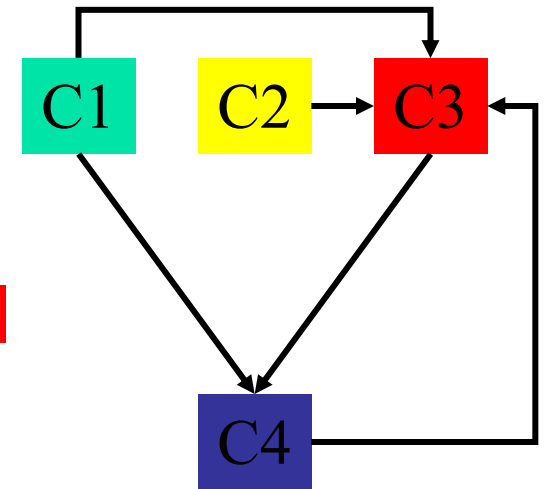
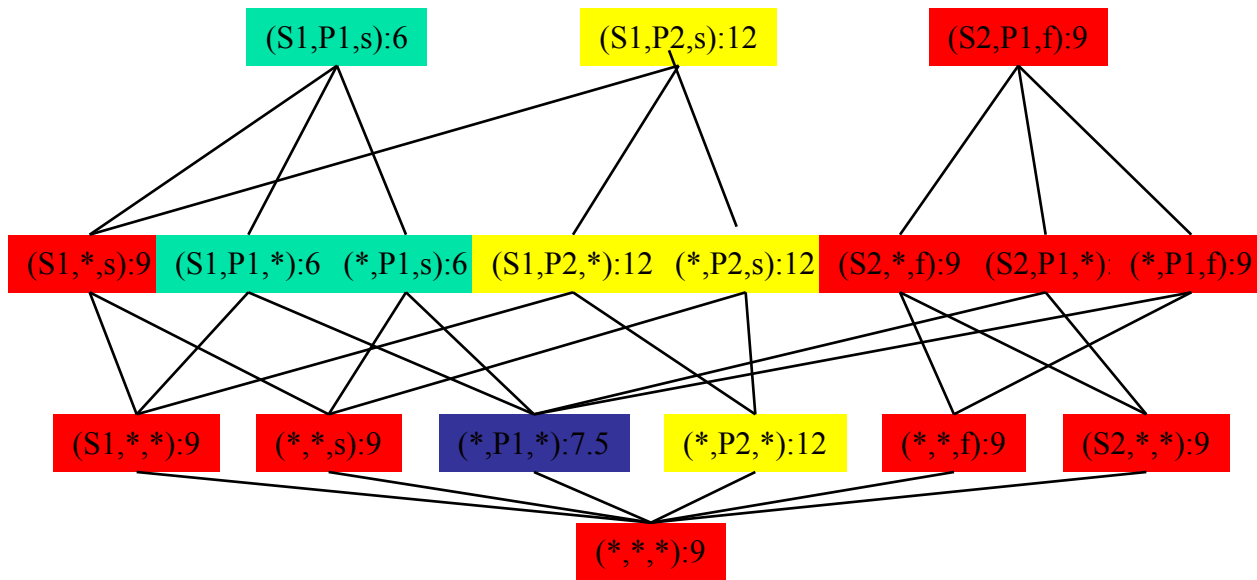
# Quotient Cube

- 很多cells具有相同的聚集值
- 是否可以根据聚集值将cells分组, 从而总结出cube的一些语义信息?



# 原始方法

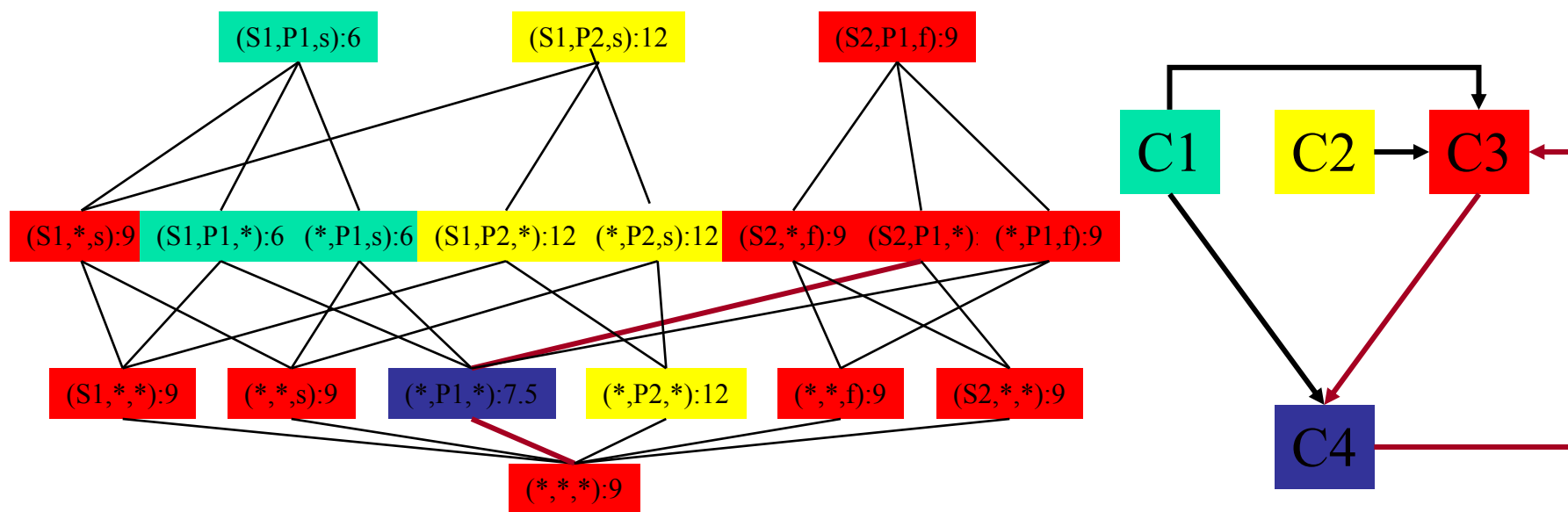
- 将Cube进行划分，具有相同聚集值的cells被分在同一个类中



# 原始方法所具有的问题

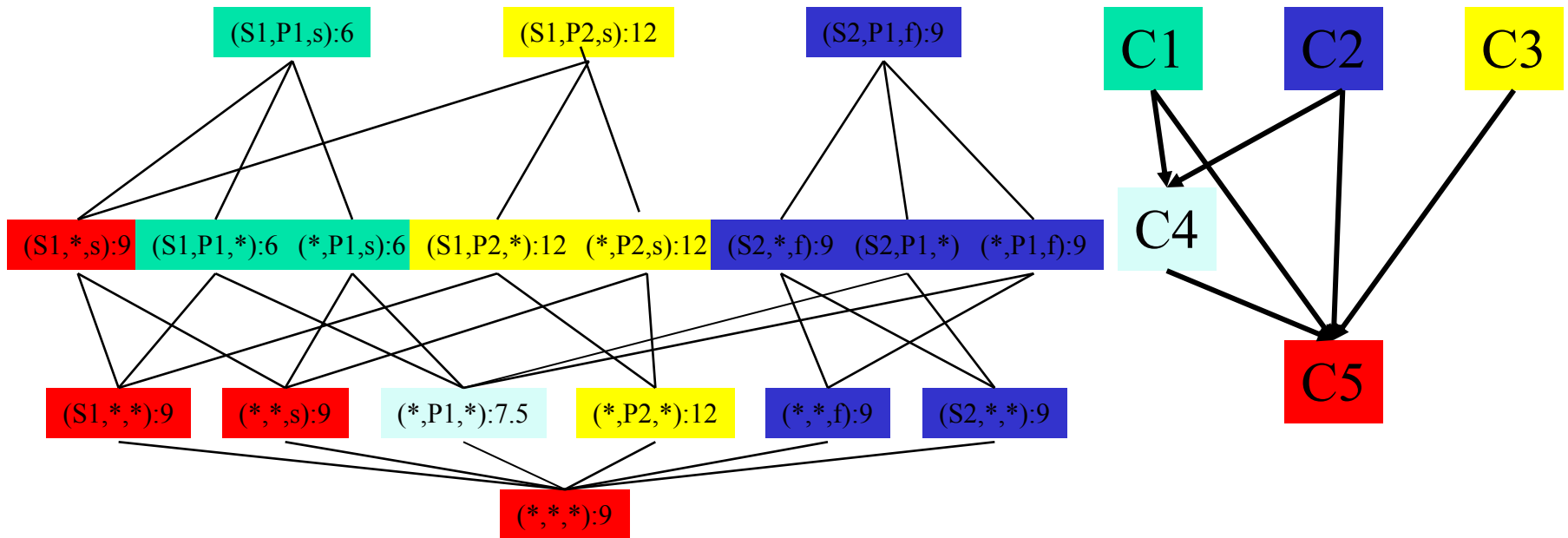
- 划分后，不再是一个格，丢失了上卷/下钻等语义信息

$$C3 \xrightarrow{\text{rollup}} C4 \xrightarrow{\text{rollup}} C3$$



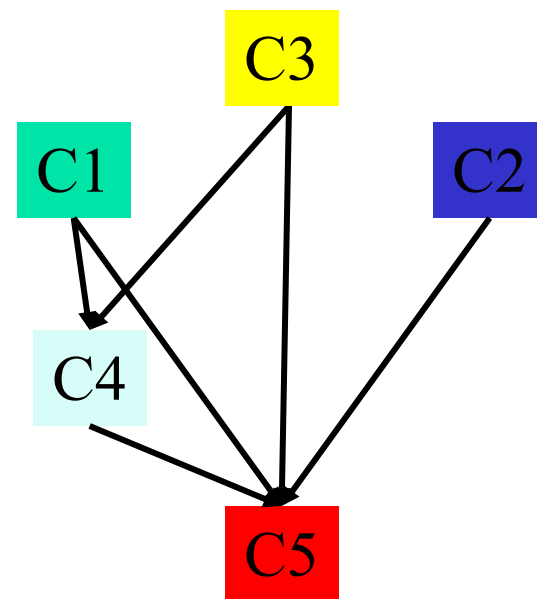
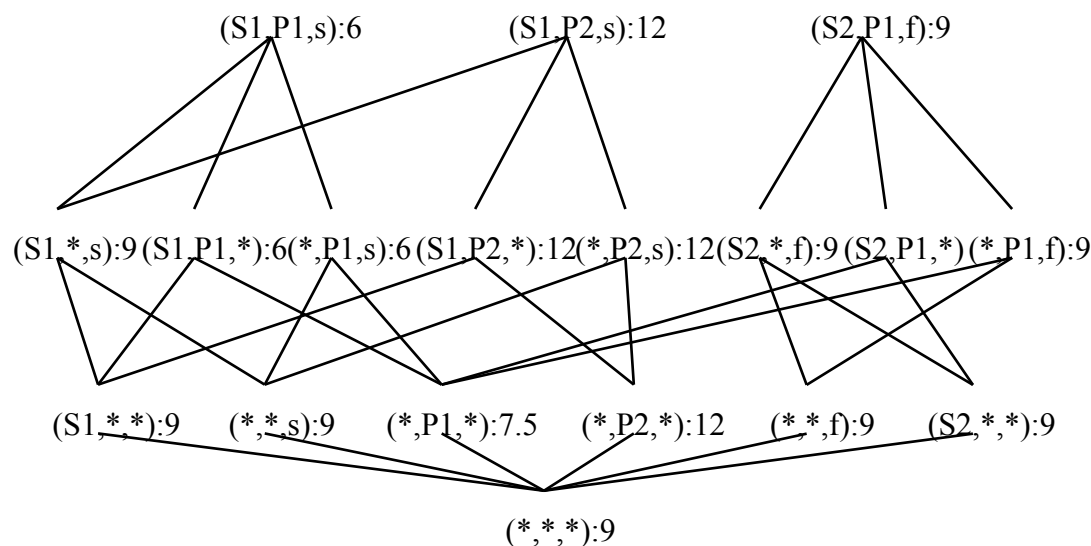
# 改进方法：等价划分

- **Quotient cube**: 在划分的时候保留上卷/下钻等语义信息



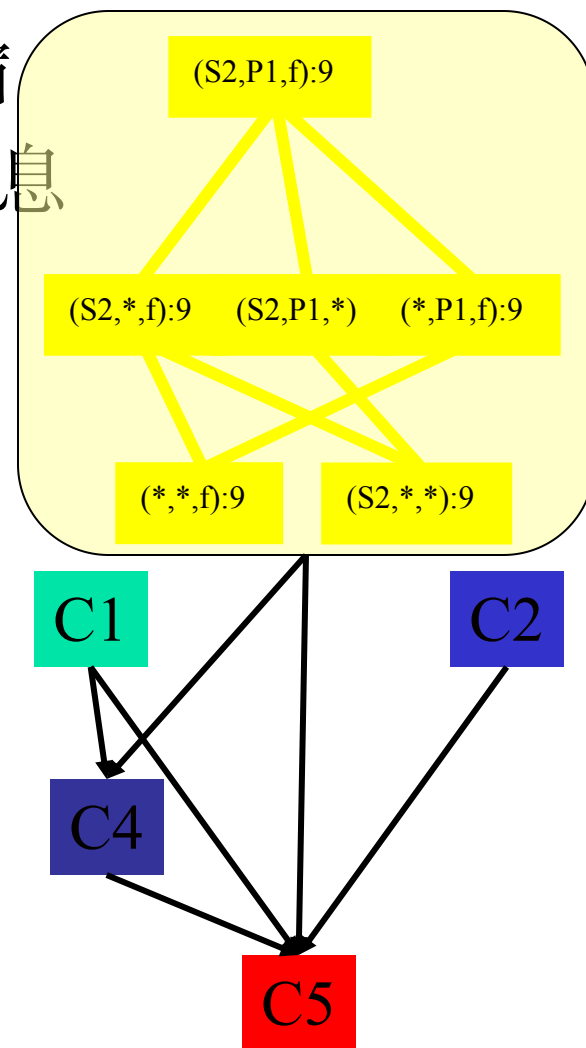
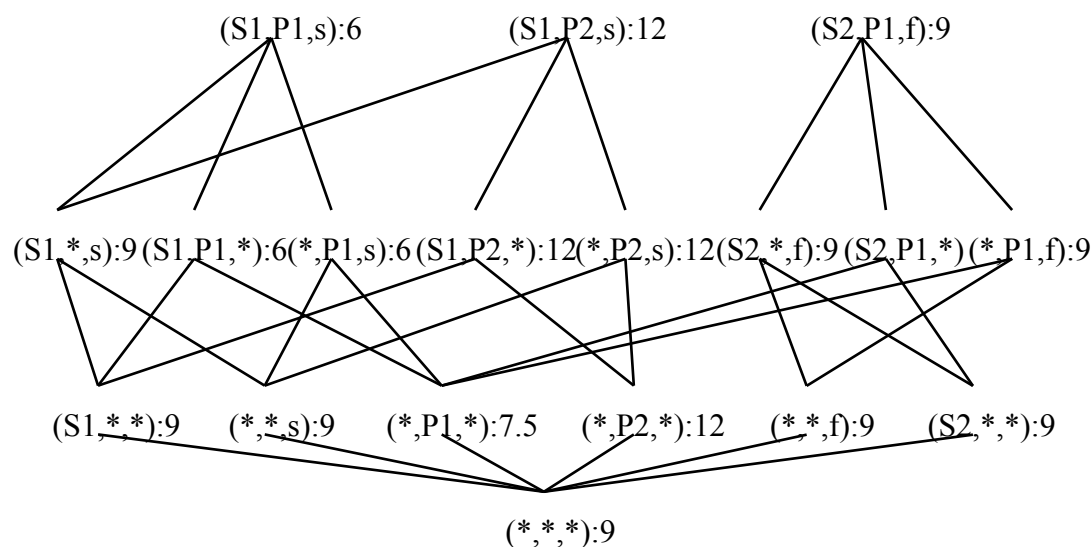
# Quotient Cube好在哪？

- 压缩比大，且查询时不需要解压缩
- 保留了OLAP的上卷/下钻等语义信息



# Quotient Cube好在哪？

- 压缩比大，且查询时不需要解压缩
- 保留了OLAP的上卷/下钻等语义信息



# Close原理

---

Quotient数据方体在对所有的数据元素进行等价划分的时候，利用了一个重要的原理：**Close原理**。

接下来我们首先介绍什么是**Close原理**

# Close原理

---

**Close**原理最早用于关联规则的挖掘算法中，用来找出最具代表性的频繁项目集。其基本思想是求两个集合之间的最大匹配。

例如，在购物篮事务数据集中，存在两个集合，一个是项目集合 $I=\{i_1, i_2, \dots, i_m\}$ ,另一个是事务集合 $T=\{t_1, t_2, \dots, t_n\}$ 。如果利用**Close**原理，求出最具代表性的频繁项目集，则其他的频繁项目集可以由该频繁项目集推出。从而减少了所求出的频繁项目集的个数以及由它们所产生的规则的个数。

# Close原理

---

首先来定义两个操作:

- 设 $X \subseteq I$ , 操作 $T(X) = \{ y \in T \mid \forall x \in X (x, y) \in DB \}$ , 根据给定的项目集, 求包含该项目集的所有事务。
- 设 $Y \subseteq T$ , 操作 $I(Y) = \{ x \in I \mid \forall y \in Y (x, y) \in DB \}$ , 根据给定的事务集, 求同时出现在该事务集合中的所有的项目

这两个操作互为逆操作。

# Close原理

例如，假设有下面六条购物篮事务，最小支持度是3:

- 1 ACTW
- 2 CDW
- 3 ACTW
- 4 ACDW
- 5 ACDTW
- 6 CDT

如果 $X = ACW$  的话，则 $T(X) = \{1345\}$ 。

如果 $Y = 245$ ，则 $I(Y) = CDW$ 。

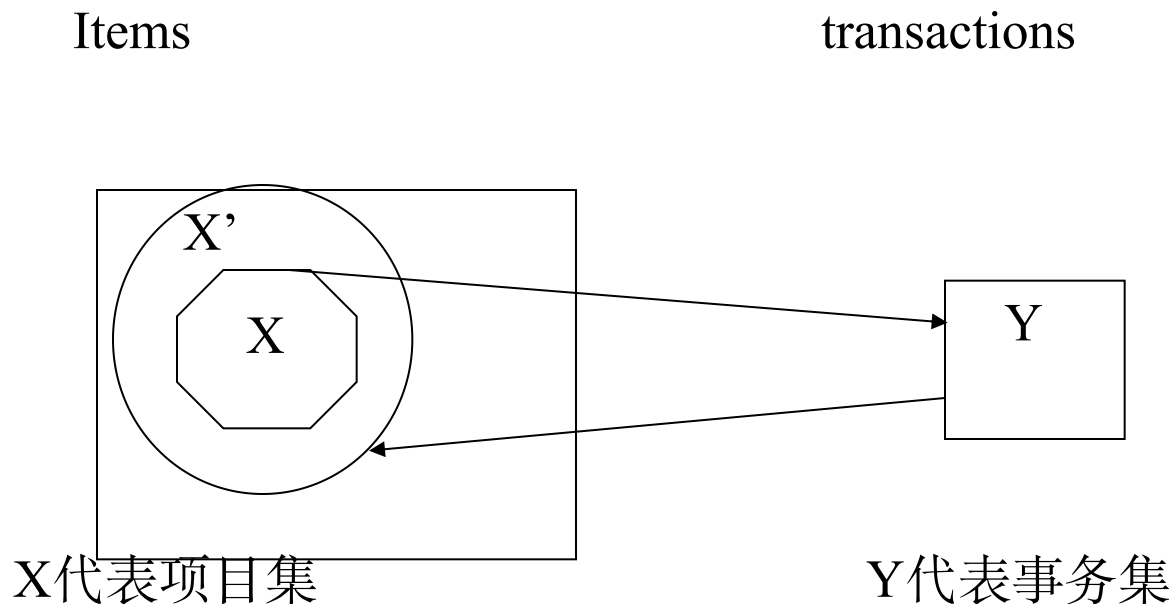
如果 $X = X'$ ，我们则说 $X$ 是close的。

上例中 $I(T(x)) = X' = ACW$ ,则说项目集合 $ACW$ 是close的。

另例：设  $X = AT$ ，则 $T(X) = 135$ ,  $I(T(X)) = ACTW$ , 项目集 $ACTW$ 是close的。

# Close原理

从上述两个例子可以看出，经过如下图所示的一次循环操作之后，我们总能得到**close**的频繁项目集。



# Close原理

---

Quotient数据方体将data cube中的所有cells看作一个集合，将事实表中的所有的tuples看作一个集合，在这两个集合之上应用close原理，将所有的cells进行等价划分。

划分后，每一个等价类中的所有的cells都从相同的tuple聚集而来，因此它们具有相同的聚集值，可以压缩存储在同一个物理单元中。这样，就可以大大地缩减数据方体的空间。